2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion) | 979-8-3315-3683-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSE-COMPANION66252.2025.00079

CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph

Hanxiang Xu*‡, Wei Ma*§, Ting Zhou‡, Yanjie Zhao†‡, Kai Chen‡, Qiang Hu¶, Yang Liu‖ and Haoyu Wang‡
‡ Huazhong University of Science and Technology, Wuhan, China
xuhx@hust.edu.cn, tingzhou27@hust.edu.cn, yanjie_zhao@hust.edu.cn, kchen@hust.edu.cn, haoyuwang@hust.edu.cn
§ Singapore Management University, Singapore, weima@smu.edu.sg
¶ The University of Tokyo, Tokyo, Japan, qianghu0515@gmail.com
‖ Nanyang Technological University, Singapore, yangliu@ntu.edu.sg

Abstract*—In recent years, the programming capabilities of large language models (LLMs) have garnered significant attention. Fuzz testing, a highly effective technique, plays a key role in enhancing software reliability and detecting vulnerabilities. However, traditional fuzz testing tools rely on manually crafted fuzz drivers, which can limit both testing efficiency and effectiveness. To address this challenge, we propose an automated fuzz testing method driven by a code knowledge graph and powered by an LLM-based intelligent agent system, referred to as CKGFuzzer. We approach fuzz driver creation as a code generation task, leveraging the knowledge graph of the code repository to automate the generation process within the fuzzing loop, while continuously refining both the fuzz driver and input seeds. The code knowledge graph is constructed through interprocedural program analysis, where each node in the graph represents a code entity, such as a function or a file. The knowledge graph-enhanced CKGFuzzer not only effectively resolves compilation errors in fuzz drivers and generates input seeds tailored to specific API usage scenarios, but also analyzes fuzz driver crash reports, assisting developers in improving code quality. By querying the knowledge graph of the code repository and learning from API usage scenarios, we can better identify testing targets and understand the specific purpose of each fuzz driver. We evaluated our approach using eight open-source software projects. The experimental results indicate that CKGFuzzer achieved an average improvement of 8.73% in code coverage compared to state-of-the-art techniques. Additionally, CKGFuzzer reduced the manual review workload in crash case analysis by 84.4% and successfully detected 11 real bugs (including nine previously unreported bugs) across the tested libraries. Our research enhances the overall performance of fuzz testing by refining fuzz driver generation strategies and input seed analysis, offering a more effective solution for vulnerability remediation and software quality improvement.

I. Introduction

Since the emergence of Large language models (LLMs), their remarkable programming capabilities have attracted significant attention from researchers.

By leveraging the code generation capabilities of LLMs, researchers are exploring solutions to more complex programming challenges [9], such as software self-repair and code translation. Among these, fuzzing has gained significant attention due to its prominent role in identifying software vulnerabilities and enhancing software reliability. Consequently, an increasing number of studies are utilizing LLMs to further improve the performance of fuzzing in order to better address these challenges. Although traditional fuzzing tools, such as AFL [7] and libFuzzer [26], are well-established, the creation of fuzz drivers still relies heavily on manual coding, which limits the efficiency and scope of testing. As software systems grow in size and complexity, manually writing fuzz drivers becomes increasingly time-consuming and prone to overlooking potential test scenarios. Given that the effectiveness of fuzz testing is closely tied to the quality of the fuzz drivers, low-quality drivers can significantly weaken the effectiveness of the tests. Additionally, the input seeds used in fuzz testing play a crucial role in determining the success of the process.

Existing approaches, such as PromptFuzz [27], generate fuzz drivers using LLMs through different combinations of API mutations. These methods employ zero-shot or few-shot learning techniques, such as Google OSS-Fuzz-Gen[1], to generate fuzz drivers based on a given set of APIs. Zhang et al. [45] have explored various generation strategies as well. However, these methods have not fully addressed the challenges associated with generating high-quality fuzz drivers, resulting in limitations in both the quality and effectiveness of the generated drivers. For example, while APIs in libraries can be called independently, they often have specific use cases, and different functional requirements may necessitate combining various APIs. Recognizing these combinations is crucial for developers when analyzing crashes, as it helps trace the source of errors and assess their impact, ultimately improving the efficiency of vulnerability fixes and software quality. Moreover, fuzz driver generation differs from unit testing; it not only requires an understanding of individual functions but also the interactions between multiple APIs, demanding deep insight into system code behavior. Without sufficient contextual information, LLMs struggle to generate high-quality fuzz drivers.

*Hanxiang Xu and Wei Ma contributed equally to this work.
†Yanjie Zhao is the corresponding author (yanjie_zhao@hust.edu.cn).
‡The full name of the author's affiliation is Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology.

[1] https://github.com/google/oss-fuzz-gen

2574-1934/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSE-Companion66252.2025.00079

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

Another key challenge lies in fuzzing's heavy reliance on input seeds. To produce more meaningful seeds, data flow analysis on the fuzz driver is required. However, achieving this with static or dynamic analysis tools increases engineering complexity. Crash analysis presents another challenge, particularly in distinguishing between API misuse and library-level errors. Crashes may stem from the incorrect fuzz driver, improper API usage, or deeper system issues, making it challenging for developers to accurately trace the root cause. This uncertainty complicates debugging and may hinder effective vulnerability resolution. Additionally, LLM-based crash analysis depends on comprehensive contextual information and up-to-date vulnerability knowledge, which are not always readily available, further complicating issue identification.

To address these challenges, we approach fuzz driver generation as a code generation task. By constructing a knowledge graph based on code repositories, we use LLMs to automatically generate fuzz drivers and perform data flow analysis to produce input seeds, thereby achieving fully automated fuzz testing. To enable LLMs to better understand the software under test, we propose abstracting the codebase into a code knowledge graph. In this graph, nodes represent code elements (such as functions, classes, and modules), while edges depict their interactions, with each node containing detailed information about the element's functionality and behavior.

We treat fuzz driver generation as a code planning task, beginning with the formulation of a generation plan followed by iterative fuzz driver generation. To this end, we developed an LLM-driven fuzz driver agent system, CKGFuzzer, based on the code knowledge graph. Our approach fully automates the fuzzing process, significantly reducing engineering complexity while improving the usability of fuzz testing. The code knowledge graph provides deeper insights into code behavior, guiding the LLM in generating higher-quality API combinations for fuzz drivers and coverage-guided mutation.

We evaluated our approach using eight open-source libraries and addressed three research questions (RQs). First, we compared CKGFuzzer with other generation techniques (RQ1). Second, we analyzed the influence of each agent in CKGFuzzer (RQ2). Third, we investigated whether CKGFuzzer can effectively analyze the causes of crashes generated during fuzzing (RQ3). Our experiments demonstrate that CKGFuzzer, supported by the code knowledge graph, can generate high-quality fuzz drivers with enhanced code coverage. Through extensive evaluations across these libraries, we found that the agents of CKGFuzzer play a crucial role in fuzz driver generation. Additionally, our analysis of crash reports indicates that CKGFuzzer can provide valuable insights for developers.

In summary, our work has three main contributions:

1) We propose CKGFuzzer, which leverages a code knowledge graph to enhance fuzz driver generation. By analyzing the relationships between code elements and APIs, CKGFuzzer automatically generates high-quality fuzz drivers that achieve better code coverage.
2) CKGFuzzer introduces a fully automated fuzzing framework that integrates multi-agent systems and LLMs. It automates key steps such as fuzz driver generation, input seed initiation, dynamic program repair, and coverage-guided mutation, significantly reducing manual effort and enhancing overall efficiency in the fuzzing process.
3) Experimental results show that CKGFuzzer achieved an average increase of 8.73% in code coverage compared to the state-of-the-art methods. Furthermore, CKGFuzzer reduced the manual review in crash case analysis by 84.4% and successfully identified 11 real bugs (including nine previously unreported bugs) of tested libraries.

Artifact Availability. We make our artifact public available at https://github.com/security-pride/CKGFuzzer.

II. Background and Related Work

A. Fuzz Testing

Fuzzing is a technique used to detect software security vulnerabilities [46]. It works by strategically mutating input data and executing test cases while monitoring the execution process, thereby effectively identifying defects. However, despite the high efficiency of fuzz testing in detecting flaws, it faces a significant challenge: the input space is vast, while the inputs that can trigger defects are exceedingly rare. As a result, the core of fuzzing lies in how to efficiently identify these sparse inputs that may cause software vulnerabilities. To increase the likelihood of discovering defects, fuzzers often use execution feedback (such as execution states or outcomes) as a fitness metric, with code coverage being a common example. Depending on the level of analysis performed on the system under test, fuzzing techniques can be categorized into three types [20]: black-box, white-box, and gray-box. Black-box fuzzing depends solely on external observations without internal insights, while white-box fuzzing requires detailed analysis of the source code, with gray-box fuzzing positioned as an intermediate approach between the two.

Fuzzing has a wide range of applications [29], [30], [40], [17], including grammar-based test case generation, kernel testing, GUI testing and penetration testing. One of the most well-known and versatile fuzzing tools is American Fuzzy Lop (AFL). Additionally, there are tools tailored to specific domains, such as Bbuzz [2], which focuses on network protocol testing, and Ffuf [37], designed for web-based testing. Library API fuzzing is an important application area, focusing on testing the security of library interfaces. Compared to general fuzzing methods, Library API fuzzing targets library interfaces specifically, aiming to uncover potential vulnerabilities in how they handle exceptional inputs. This method is commonly used to test libraries that are widely used across different applications, ensuring their security and stability under various input conditions. LibFuzzer is a widely-used tool for testing library APIs, generating large volumes of random input to evaluate the robustness of library interfaces. Several other tools also integrate code analysis techniques to enhance fuzzing. For example, RULF [13] tests Rust libraries by analyzing API dependency graphs, ensuring safe operation in complex dependency contexts, while GraphFuzz [8] leverages data flow analysis for API testing to uncover potential security risks.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

B. LLM for Fuzzing

The integration of LLMs with fuzzing has received significant attention in recent research, particularly in the context of generating more intelligent and context-aware test cases for software systems [11], [41], [14]. Traditional fuzzing techniques often struggle with generating meaningful inputs for complex software systems. LLMs have shown great promise in addressing these limitations by enabling more sophisticated fuzzing strategies with powerful generative capabilities. In recent years, several LLM-based fuzzers have been proposed to improve the efficiency and effectiveness of fuzzing techniques. For example, LLMs have been used in tools like TitanFuzz [3], FuzzGPT [4], WhiteFox [43] and ParaFuzz [42], which leverage models such as GPT [1], Codex [6], and other generative models to create diverse and high-quality test inputs. These fuzzers introduce LLMs into prompt engineering and seed mutation to enhance fuzzing performance.

LLMs are also used to automate the fuzzing process. InputBlaster [23] employs LLMs to generate specialized text inputs for mobile applications, significantly increasing bug detection rates compared to traditional methods. Similarly, ChatAFL [31] and ChatFuzz [10] integrate LLMs to enhance the fuzzing of network protocols and web applications, resulting in higher code coverage and vulnerability detection rates. Another important aspect of LLM-based fuzzing is the mutation of seed inputs. In traditional fuzzing, mutation strategies such as bit-flipping and block-based mutations are used to generate new test cases [39]. However, LLM-based fuzzing can perform more sophisticated mutations by understanding the context of the input, making them particularly effective for generating valid yet diverse test cases. For example, CovRL-Fuzz uses LLMs to perform mutation operations, guided by coverage feedback, ensuring that mutations not only maintain grammatical correctness but also increase coverage, thus making it more likely to trigger unexpected software behavior [5].

In addition to improving traditional fuzzing techniques, LLMs have increasingly been employed to automate the generation of fuzz drivers, which are essential for targeting specific APIs or software functions during the fuzzing process. Zhang et al. [44] demonstrate the use of GPT-3.5 and GPT-4 to automatically generate fuzz drivers for complex library APIs, achieving an automation rate of over 60%. This automation significantly reduces the manual effort required to create fuzz drivers and helps streamline the fuzz driver generation process, which is typically a bottleneck in fuzzing campaigns, particularly for large and complex software systems. Similarly, Lyu et al. [27] propose PromptFuzz, a coverage-guided fuzzer that leverages LLMs to iteratively generate fuzz drivers capable of exploring previously undiscovered library code.

III. Methodology

A. Overview

In this section, we present the system design of CKGFuzzer, a fuzz driver generation framework that integrates a multi-agent system with a code knowledge graph. CKGFuzzer is designed to enhance the quality and coverage of fuzz testing by efficiently generating fuzz drivers for API combinations. The primary objective of our framework is to harness the rich and relative information within the code knowledge graph to capture the complex invocation relationships among library APIs and, based on this, generate highly effective fuzz drivers. By introducing a coverage-guided mutation strategy, CKGFuzzer continuously optimizes API combinations, enabling the exploration of additional code paths and the identification of hidden vulnerabilities. Figure 1 illustrates the workflow of CKGFuzzer, which comprises three main components and a unique feedback loop mechanism.

Initially, CKGFuzzer parses the target project and its library APIs, extracting and embedding a code knowledge graph from the project under test. This parsing process involves two steps: first, parsing the abstract syntax tree, and second, performing interprocedural program analysis. The parser extracts key information such as data structures, function implementations, function signatures, and calling relationships. Next, CKGFuzzer queries API combinations for each library API, focusing on those that have invocation relationships or functional similarities, and generates corresponding fuzz drivers. CKGFuzzer then attempts to compile these generated fuzz drivers and resolves the compilation errors that arise. During the driver generation and repairing, we provide CKGFuzzer with a dynamically updated knowledge base on library usage. Once the fuzz drivers are successfully compiled, CKGFuzzer executes them while monitoring the code coverage of each library file. It employs a code coverage-guided iterative process to mutate API combinations that fail to cover new paths. This iterative process continues until either new code paths are discovered or the mutation budget is exhausted. Finally, CKGFuzzer uses chain-of-thought reasoning to analyze any crashes produced during fuzzing. To verify the validity of these crashes, we reference an LLM-generated knowledge base containing real CWE-related source code examples.

B. Construction of the Code Knowledge Graph

We begin by employing a syntax parser and static analysis tool to extract detailed information on functions, file structures, and function call relationships from the code repository. In addition, we gather API function details and relevant documentation from existing repository resources. To further enhance understanding, we utilize an LLM to generate concise code summaries, usage scenarios, and file-level overviews.

We now introduce the symbols used to describe the code knowledge graph. Let the set of functions in the codebase be

$$\mathcal{F} = \{f_1, f_2, \ldots, f_N\},$$

where each function f_i has the following attributes: source code c_{f_i}, signature σ_{f_i}, associated file d_{f_i}, and function summary s_{f_i}.

The set of files in the codebase is

$$\mathcal{D} = \{d_1, d_2, \ldots, d_M\},$$

where each file d_j have a summary s_{d_j}.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

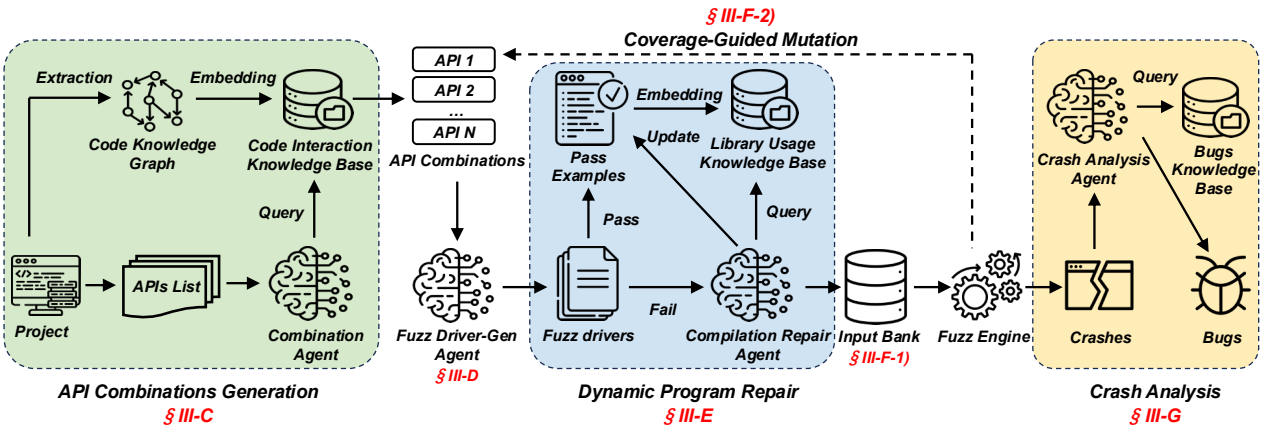

Fig. 1: Overview of CKGFuzzer

The extracted call relationships are denoted as

$$\mathcal{C} = \{(f_i, f_j) \mid f_i \text{ calls } f_j\}.$$

We define a mapping from functions to their summaries as

$$S : f_i \mapsto s_{f_i}.$$

The construction process proceeds as follows:

a) Initialize Knowledge Graph G: We first create an empty property graph $G = (V, E)$, where V represents the set of nodes and E represents the set of edges.

b) Create Function Nodes: For each function $f_i \in \mathcal{F}$, we create a function node $v_{f_i} \in V$, with attributes including its signature σ_{f_i}, file path d_{f_i}, source code c_{f_i}, and summary s_{f_i}.

c) Create File Nodes: For each file $d_j \in \mathcal{D}$, we create a file node $v_{d_j} \in V$ with attributes such as the file summary s_{d_j}. For each function $f_i \in \mathcal{F}$ where $d_{f_i} = d_j$, we create an edge $e_{d_j,f_i} = (v_{d_j}, v_{f_i}) \in E$ labeled as "CONTAINS".

d) Create Call Relationship Edges: For each call relationship $(f_i, f_j) \in \mathcal{C}$, if both f_i and f_j belong to $\mathcal{F}$, we create an edge $e_{f_i,f_j} = (v_{f_i}, v_{f_j}) \in E$ labeled as "CALLS". If f_j is an external library function, we create an external function node v_{f_j}, labeled as "LIBRARY_FUNCTION", and create an edge $e_{f_i,f_j} = (v_{f_i}, v_{f_j}) \in E$ labeled as "LIBRARY_CALLS".

e) Build Property Graph Index: We utilize multiple forms of graph indexing, including code indexing and natural language indexing. Since code and natural language reside in different embedding spaces, relying on one type of index for queries of the other type may lead to inaccurate retrieval. Therefore, we adopt separate indexes for each data format: natural language and code. We create two types of property graph indexes: $PGI_{NL} = (G, \mathcal{V}_{NL})$ for natural language and $PGI_{code} = (G, \mathcal{V}_{code})$ for code, ensuring efficient querying.

C. API Combination Generation

The selection of API combinations is a critical step in guiding the generation of fuzz drivers, as cleverly combining APIs within a single driver can reveal more unexplored execution paths within the target library. CKGFuzzer employs an external retrieval-augmented process, combining code knowledge graphs with LLM to generate optimal API combinations, which are then used to create fuzz drivers that maximize code coverage. As shown in Figure 1, the CKGFuzzer framework follows a multi-step method for API combination generation.

This generation process involves querying the code knowledge graph to retrieve relevant API combinations based on a target API. The query process is systematically divided into three key stages: Indexing, Retrieval, and Response Synthesis, as illustrated in Algorithm 1.

Algorithm 1: Query for API Combinations

Input: Query Q, Knowledge graph G, Embedding model M, Similarity threshold s, Number of chunks k, LLM L, Refine prompt P_r
Output: API combination C

$IndexBase \leftarrow \texttt{Embed}(G, M)$;
$Q_{vec} \leftarrow \texttt{Embed}(Q, M)$;
$C_1, C_2, \ldots, C_k \leftarrow \texttt{RetrieveChunks}(Q_{vec}, IndexBase, s, k)$;
$R \leftarrow \texttt{LLM}(Q, C_1)$;
foreach *remaining chunk* C_i *in* $C_2, \ldots, C_k$ **do**
  $R \leftarrow \texttt{RefineWithLLM}(R, C_i, P_r, L)$;
$C \leftarrow \texttt{LLM}(Q, R)$;
return C;

In the indexing phase, the knowledge graph is transformed into an index base by splitting it into smaller, manageable chunks. Each chunk represents a subset of the graph's content, allowing the system to efficiently retrieve relevant information during the query process. The indexing ensures that the knowledge graph is well-organized for querying by the LLM.

When the LLM issues a query, representing a request for APIs related to a target API, the system retrieves a set number

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

of chunks from the indexed knowledge graph. The selection of these chunks is based on their relevance to the query, ensuring that the most pertinent information is retrieved to form a response.

The retrieved chunks are processed sequentially and carefully, with the final response being synthesized through an iterative and gradual refinement process [28]. The initial chunk serves to initialize the query answer, prompting the LLM to iteratively incorporate the useful remaining chunks while simultaneously filtering out noisy or irrelevant data. This optimization process continues progressively until all retrieved information has been thoroughly considered. The final output consists of a set of APIs that are functionally related to the target API. Through this iterative refinement approach, all useful and relevant information is effectively integrated into the final response, minimizing the impact of noisy data.

D. Fuzz Driver Generation

To generate high-quality fuzz drivers, we adopt a role-prompt strategy combined with a memory mechanism to guide the LLMs [18], [19]. These prompts are constructed using both the source code and natural language descriptions of the API combinations, ensuring that the LLM generates fuzz drivers that are not only syntactically correct but also contextually aligned with the APIs' functionality.

Our prompt strategy emphasizes several key aspects for fuzz driver generation:

- **Task Definition**: The LLM is tasked with generating a fuzz driver that tests the provided API combination. The prompt specifies that each API must be invoked within the function `LLVMFuzzerTestOneInput`, ensuring comprehensive testing coverage of the API set.
- **API Context**: The prompt includes the API source code, headers, and natural language summaries. This provides the LLM with the necessary context to correctly utilize the APIs and manage their inputs and outputs according to their functionality and constraints.
- **Error Handling**: To prevent instability in the fuzz driver itself, which could lead to crashes and reduce overall fuzzing efficiency, the prompt includes explicit instructions for error handling and memory management.

By constructing the prompt with these components, we ensure that the generated fuzz drivers are both executable and aligned with the best practices of fuzz testing. The use of the memory mechanism allows the LLM to refine its generated response iteratively, improving the quality of the fuzz driver as it processes additional information from the API combination.

E. Dynamic Program Repair

Due to hallucinations and a lack of high-quality training data, code generated by LLMs often contains syntax and semantic errors [22], [32], [34], which may affect the reliability of fuzz drivers. A robust fuzz driver must ensure correctness at the code level; however, we observed that some fuzz drivers produced by CKGFuzzer fail to compile. These failures are primarily caused by syntax errors and incorrect usage of library APIs. To address this issue, we developed an LLM-based dynamic program repair mechanism that automatically corrects these errors and improves the overall stability of fuzz drivers, as shown in Algorithm 2. Compared to traditional static rule analysis, LLM-based solutions excel at handling complex, context-dependent errors that are difficult to resolve through static rules alone [33], [15], [16].

Algorithm 2: Fuzz Driver Repair

Input: Fuzz driver list D_{list}, API usage samples S, Max iterations M, LLM L
Output: Repaired drivers D_{rep}, Knowledge base KB

```
D_rep ← ∅
KB ← InitKB(S);
foreach D ∈ D_list do
    i ← 0; compiled ← false
    while i < M and not compiled do
        result, errors ← Compile(D)
        if result is success then
            compiled ← true; D_rep ← D_rep ∪ {D}
            KB ← UpdateKB(KB, D)
        else
            query ← ConstructQuery(errors)
            correct_cases ←
              QueryKB(L, KB, query)
            D ← LLMRepair(L, D, correct_cases)
        end
        i ← i + 1
    end
end
return D_rep, KB
```

To implement dynamic program repair, we first initialize a knowledge base containing correct library API usages, constructed from samples of fuzz drivers from OSS-Fuzz and the library's header files. It should be noted that while the OSS-Fuzz code focuses on individual APIs, our approach handles API combinations. The header files help resolve errors caused by missing include statements. When CKGFuzzer encounters a compilation failure, it processes the compiler error messages and constructs a query to search the external knowledge base for the correct usage of the erroneous API or code snippet. CKGFuzzer continues to query the knowledge base and apply fixes iteratively until the fuzz driver successfully compiles or a maximum iteration limit is reached.

Additionally, all successfully compiled fuzz drivers are inserted back into the knowledge base, dynamically updating the repository of correct API usage patterns. This dynamic update strategy ensures that the knowledge base evolves over time, gradually expanding to cover a wider range of API usage scenarios across different libraries. As a result, our approach maintains a high level of adaptability and accuracy in fixing errors, reducing the chances of recurrent compilation failures.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

F. Fuzzing Loop

The fuzzing loop in CKGFuzzer is specifically designed to iteratively improve the overall effectiveness of fuzz testing by carefully initializing high-quality inputs and strategically guiding mutations based on detailed coverage feedback. The loop primarily consists of two main components: the input bank and the coverage-guided mutation mechanism, both of which work closely together to effectively maximize code coverage and systematically identify new execution paths.

1) Input bank initialization: Before fuzzing begins, we initialize an input bank by generating input seeds for each fuzz driver using LLMs. The input generation process starts with a detailed analysis of the fuzz driver's data flow, where we extract the value flow relationships between variables. Subsequently, we extract the API function signatures that the fuzz driver interacts with, which helps in understanding the structure and constraints of the expected inputs.

Using this information, we construct a prompt for the LLM, instructing it to generate input seeds that maximize code coverage, target edge cases, and explore boundary conditions. The generated inputs are designed to adhere to the data flow dependencies and the API's input requirements, ensuring that they are both valid and effective in triggering diverse execution paths from the very outset of fuzzing.

The initial input seeds stored in the input bank serve as a foundation for the fuzzing process. As fuzzing progresses, the input bank is continuously updated with new inputs generated from coverage feedback, ensuring that the fuzzing loop remains dynamic and adaptive.

2) Coverage-guided mutation: During the fuzzing process, CKGFuzzer carefully monitors the code coverage achieved by each fuzz driver. It thoroughly analyzes the coverage data across the entire library and identifies files with lower coverage compared to the overall library. For each low-coverage file, the defined API functions are extracted, and a prioritized list, known as the low coverage API list, is generated based on the detailed coverage rankings of the files.

This low coverage API list is then used to guide the LLM in mutating and restructuring the current API combinations in the fuzz driver. The primary objective is to explore new execution paths and increase code coverage by specifically focusing on under-explored APIs. As clearly shown in Algorithm 3, the mutation process is iterative: CKGFuzzer continues to query the LLM with new API combinations based on the low coverage API list until new branches are thoroughly covered or computational resources are exhausted.

These mutation strategies are guided by the coverage feedback from previous executions. By strategically prioritizing low-coverage APIs, CKGFuzzer consistently focuses on areas of the code that are less explored, thereby maximizing the likelihood of discovering new program behaviors.

Algorithm 3: Coverage-Guided Mutation

Input: Library F, Current API combination A, Current library coverage $C_{current}$, LLM L, Max iterations M
Output: Updated API combination A_{new}

```
lowCovFiles ← {};
lowCovAPIs ← {};
foreach file ∈ F do
    fileCoverage ← AnalyzeFileCoverage(file, A);
    if fileCoverage < C_current then
        lowCovFiles ← lowCovFiles ∪ {file};
        apis ← ExtractAPIInfo(file);
        lowCovAPIs ← lowCovAPIs ∪ apis;
sortedLowCovAPIs ← SortByCoverage(lowCovAPIs, lowCovFiles);
i ← 0;
if_newPath ← false;
while i < M and not newBranch do
    query ← ConstructQuery(sortedLowCovAPIs);
    A_new ← MutateAPICombination(L, A, query);
    if_newPath ← DetectNewPath(F, A_new);
    i ← i + 1;
return A_new
```

G. Crash Analysis

During the fuzzing process, CKGFuzzer is equipped with several runtime error detection tools, like AddressSanitizer [35] (ASan), Undefined-Behavior-Sanitizer [25] (UBSan) and MemorySanitizer (MSan) [38]. However, the crashes detected by these tools can originate from different sources, including incorrect API usage in the fuzz driver or bugs within the library APIs themselves. To distinguish between these causes, we developed an automatic crash analysis module designed to identify the root cause of each crash.

The crash analysis module employs a chain-of-thought strategy, guiding the LLM through a structured, step-by-step reasoning process to diagnose the source of each crash. This methodical approach ensures that each stage builds upon the insights gained from the previous one, leading to a clear understanding of the crash's root cause. The process unfolds through the following key steps:

- **Step 1: Source Code Extraction**: The analysis begins by isolating the specific code regions where the crash occurred. This includes extracting relevant portions of the fuzz driver's source code, along with any associated API calls that may have influenced the crash.
- **Step 2: Error Pattern Hypothesis**: After extracting the relevant code, the LLM systematically analyzes the code for patterns indicative of common programming errors. These patterns could include unsafe memory operations, incorrect variable assignments, improper control flow conditions and so on. At this stage, the LLM forms initial hypotheses about which code patterns might be

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

responsible for triggering the crash, setting up a list of plausible error sources for deeper exploration.

- **Step 3: CWE-Based Pattern Matching**: Armed with specific hypotheses about potential error patterns, the LLM constructs and designs targeted queries to retrieve a comprehensive CWE-based knowledge base. This extensive knowledge base, comprising over 100 real-world CWE vulnerabilities related to C/C++ programs, provides a wealth of detailed information about known issues.

By progressively narrowing down the potential causes and comparing them against known vulnerabilities, the chain-of-thought strategy ensures that the LLM can systematically trace the crash to its most likely source, whether it stems from fuzz driver errors or underlying library bugs.

The knowledge base used in the crash analysis module is constructed from a collection of real-world CWE vulnerabilities specific to C and C++ programs. Each entry in the knowledge base includes:

- A natural language description of the vulnerability, detailing its characteristics and typical causes.
- A corresponding code sample generated by the LLM based on the vulnerability description, exemplifying the vulnerability in a real-world context.

The natural language descriptions serve as prompts for the LLM to better understand the crash context. The inclusion of example code allows the LLM to draw comparisons between the source code of fuzz driver or library APIs and known vulnerabilities, improving the accuracy of the crash analysis.

IV. Evaluation

To evaluate the performance of CKGFuzzer, we defined three research questions:

RQ1: Is CKGFuzzer more effective than other fuzzers?

RQ2: How does each agent contribute to enhancing the effectiveness of CKGFuzzer?

RQ3: Can CKGFuzzer accurately analyze the cause of crashes generated during fuzzing?

A. Experimental Environment

1) Implementation: We utilize Tree-sitter[2] to parse the source code, offering an efficient method for constructing and updating the syntax tree as the source file undergoes changes. For more in-depth analysis of the codebase, such as extracting API call relationships, we employ CodeQL[3]. CodeQL enables us to query the entire repository and gather pertinent information about API usage, which serves as the foundation for the subsequent stages of processing.

To represent the extracted code and API calls in an embedding space, we evaluated several embedding models and ultimately selected the BAAI/bge-small-en-v1.5 model [12], which demonstrated strong performance in our experiments. Our approach leverages Llama-Index [24] for integrating the embeddings; however, since Llama-Index was not specifically designed for code graphs, we developed a custom RAG engine to more effectively handle code-related tasks. For fuzz driver generation and compilation-repair tasks, we chose the DeepSeek-V2-Coder model [21], primarily due to its low API pricing and its reliable performance in code generation tasks. Furthermore, we utilized the DeepSeek-V2-Chat [21] model for other agents.

All experiments were performed on a server equipped with an AMD 64-Core Processor and 1TB of RAM, running a 64-bit version of Ubuntu 22.04 LTS.

2) Settings: To evaluate CKGFuzzer, we selected a total of eight open-source libraries as our primary test subjects. We conducted the experiments using the OSS-Fuzz platform [36] and libFuzzer as the underlying fuzzing environment. We collected the API lists for each library from their respective documentation or official websites. We configured the temperature parameter for DeepSeek-V2-Coder to 0.7 and for DeepSeek-V2-Chat to 1.0. The maximum length for the default API combinations was set to 6, while the maximum number of iterations for program repair was strictly limited to 5. Additionally, the maximum number of mutations for API combinations was set to 3. To ensure that each library was fuzzed for an adequate period, we assigned different time budgets to the fuzz drivers of each library, making sure that the total fuzzing time for each library reached 24 CPU hours. Every experiment was repeated five times to mitigate statistical errors, and the average results were reported.

B. RQ1. Comparison with existing fuzzers

To address **RQ1**, we selected eight open-source libraries for evaluation: `c-ares`, `cjson`, `curl`, `lcms`, `libpcap`, `libtiff`, `libvpx`, and `zlib`. These libraries were chosen for their diverse API usage and wide adoption in various software projects. We compared CKGFuzzer with other open-source fuzzers, focusing on coverage-guided fuzzer (OSS-Fuzz [36]) and LLM-based fuzzer (PromptFuzz [27]). During the evaluation, we ran the fuzz drivers from OSS-Fuzz on each library for 24 hours. If a library in OSS-Fuzz contained multiple fuzz drivers, we allocated an equal time budget to each driver to ensure the total fuzzing time was 24 hours. For PromptFuzz, we used the same APIs list provided to CKGFuzzer and controlled the fuzzing duration to be 24 hours as well. We then evaluated the effectiveness of CKGFuzzer based on code coverage.

The code coverage results for CKGFuzzer, PromptFuzz, and OSS-Fuzz are displayed in Table I. When comparing CKGFuzzer with the other fuzzers, CKGFuzzer demonstrated the highest branch coverage in 6 out of the 8 libraries that were tested. This significant advantage can be largely attributed to the structured API combinations generated by CKGFuzzer, which are derived from a comprehensive and detailed code knowledge graph. This enables CKGFuzzer to capture the complex relationships and semantic dependencies between APIs, thereby creating more effective fuzz drivers.

In contrast, PromptFuzz initializes its API combinations through random selections and lacks a deeper understanding

[2] https://tree-sitter.github.io/tree-sitter/
[3] https://codeql.github.com/

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

TABLE I: Code coverage result of fuzz drivers generated by CKGFuzzer

Library	Version	Branches	APIs	Generated Fuzz drivers	Branch Coverage		
					CKGFuzzer	PromptFuzz	OSS-Fuzz
c-ares	1.8	6,390	119	101	**54.99%(3,514)**	48.73%(3,114)	18.79%(1,201)
cjson	1.7.18	1,379	41	41	**84.48%(1,165)**	74.11%(1,022)	38.07%(525)
curl	8.9	19,501	92	88	**26.68%(5,203)**	19.73%(3,847)	4.67%(911)
lcms	2.15	7,482	216	205	**46.93%(4,693)**	40.71%(3,046)	39.79%(2,977)
libpcap	1.10.4	5,227	67	64	**46.80%(2,446)**	39.53%(2,066)	40.81%(2,133)
libtiff	4.7	16,347	169	161	38.81%(6,344)	**48.29%(7,894)**	38.83%(6,348)
libvpx	1.14	17,640	31	29	**22.10%(3,899)**	21.09%(3,721)	15.50%(2,734)
zlib	1.3	2,375	63	61	61.6%(1,463)	**72.04%(1,711)**	51.62%(1,226)

TABLE II: Comparison of different variants across multiple libraries

Variants	c-ares		cjson		curl		lcms		libpcap		libtiff		libvpx		zlib	
	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.	Pass Rate	Cov.
Without repair	46.22% (55/119)	27.14% (1,734)	82.93% (34/41)	71.57% (987)	69.57% (64/92)	22.44% (4,376)	56.94% (123/216)	36.70% (2,746)	58.21% (39/67)	35.28% (1,844)	64.5% (109/169)	28.01% (4,586)	22.58% (7/31)	9.31% (1,642)	42.86% (27/63)	41.52% (986)
LLM-only repair	73.96% (88/119)	44.87% (2,867)	97.56% (40/41)	77.08% (1,063)	85.87% (79/92)	23.46% (4,574)	72.22% (156/216)	43.50% (3,255)	73.13% (49/67)	39.93% (2,087)	79.29% (134/169)	35.32% (5,773)	51.61% (16/31)	16.63% (2,934)	85.71% (54/63)	55.12% (1,309)
Text-only retrieval	-	47.17% (3,014)	-	79.70% (1,099)	-	22.09% (4,307)	-	40.68% (3,044)	-	40.16% (2,099)	-	36.73% (6,005)	-	21.22% (3,744)	-	55.79% (1,325)
CKGFuzzer	**84.87% (101/119)**	**54.99% (3514)**	**100% (41/41)**	**84.48% (1,165)**	**95.65% (88/92)**	**26.68% (5,203)**	**94.91% (205/216)**	**46.93% (4,693)**	**95.52% (64/67)**	**46.80% (2,446)**	**94.08% (159/169)**	**38.81% (6,344)**	**93.55% (29/31)**	**22.10% (3,899)**	**96.83% (61/63)**	**61.6% (1,463)**

of dependencies among libary APIs. As a result, CKGFuzzer outperformed PromptFuzz in libraries that require precise API usage patterns. For instance, in the `lcms` library, CKGFuzzer covered 46.93% of branches, while PromptFuzz covered only 40.71%. These results indicate that CKGFuzzer's knowledge-driven method for combining APIs enables it to explore a larger portion of the codebase, resulting in higher coverage. In cases where CKGFuzzer did not achieve the highest coverage, such as in the `libtiff` library, the gap between CKGFuzzer and PromptFuzz was relatively small (38.81% vs. 48.29%). This implies that even when PromptFuzz benefits from its constrained input mechanisms, CKGFuzzer remains competitive by leveraging its knowledge graph to generate fuzz drivers.

C. RQ2. Ablation study

In this section, we conduct experiments to investigate the impact of various components of CKGFuzzer on its effectiveness. The components we evaluate include the code knowledge graph for API combinations generation, compilation repair, and API combination mutation. Detailed experimental results are shown in Figure 2 and Table II.

1) Code knowledge graph: To evaluate the impact of the code knowledge graph on the quality of fuzz drivers generated by CKGFuzzer, we compared it with a variant that uses text-based API knowledge from libraries. This variant generates API combinations by querying a text-based knowledge base consisting of API source code and API summaries. The experimental results in Table II show that, the text-only retrieval variant achieves lower code coverage compared to CKGFuzzer, this highlights the effectiveness of the code knowledge graph in generating high-quality fuzz drivers.

2) Compilation repair: In CKGFuzzer, we introduced a dynamic program repair component to fix fuzz driver programs that encounter compilation errors. In this section, we design and carry out experiments to evaluate the effectiveness of CKGFuzzer in repairing compilation errors and the impact of the dynamic program repair component on fuzzing performance. First, we created two variants: one without any compilation error repair (`without repair`) and another that relies solely on an LLM for program repair (`LLM-only repair`). The experimental results in Table II show that the `without repair` achieved an average compilation success rate of only 57.39% (458/798), indicating that CKGFuzzer struggles to directly generate compilable fuzz driver programs. On the other hand, the `LLM-only repair` variant increased the compilation success rate to 77.19% (616/798), demonstrating the significant potential of LLMs for program repair. However, CKGFuzzer achieved a compilation success rate of 93.99% (750/798), highlighting the effectiveness of the dynamic program repair component in fixing compilation errors for fuzz driver programs.

Additionally, we evaluated the code coverage performance of the two variants. The results in Table II show that both the `without repair` and `LLM-only repair` achieved lower code coverage compared to CKGFuzzer. This is due to many fuzz drivers containing unique API combinations that failed to compile and, thus, could not participate in the fuzzing process, limiting the potential to explore new execution paths in the libraries through diverse API combinations.

3) Coverage-guided mutation: CKGFuzzer utilizes coverage-guided mutation to discover potential new API combinations. To assess its effectiveness, we conducted

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

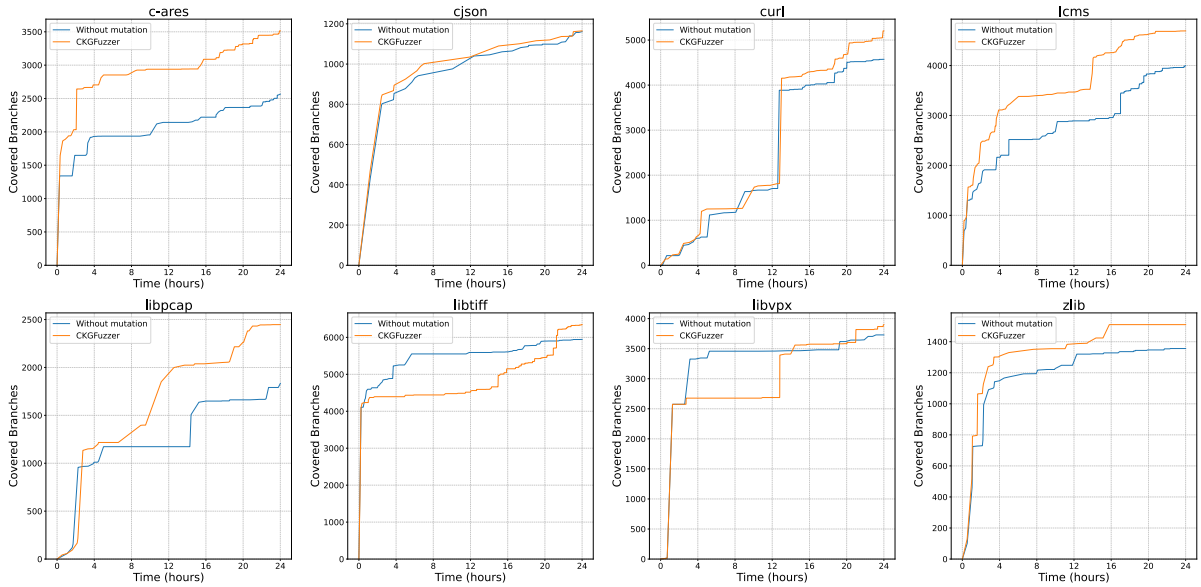

Fig. 2: Branch coverage of CKGFuzzer with and without coverage-guided mutation in different libraries

a comparative experiment against a variant that does not apply API combination mutation. We kept the fuzzing time fixed at 24 hours for both approaches to compare their coverage trends. Figure 2 shows the branch coverage results of CKGFuzzer with and without API combination mutation across different libraries.

Our coverage-guided mutation demonstrates a performance advantage across all eight libraries, although the benefit is less pronounced in `cjson` and `libvpx`. This is because the number of APIs in these two libraries is relatively small and functionally similar, meaning that nearly all possible API combinations were explored during the initial API combination generation phase. In the remaining six libraries, coverage-guided mutation significantly improved the code coverage of CKGFuzzer. The experimental results suggest that coverage-guided mutation is a more effective approach in most cases.

D. RQ3. Crash Analysis Result

This approach incorporates a dedicated crash analysis module designed to automate the detection of crash causes during fuzzing processes. As shown in Table III, we have collected and analyzed the results from this crash analysis module.

1) Overall result: CKGFuzzer identified 199 unique crashes during fuzzing, of which 168 were determined by the crash analysis module to be caused by the misuse of library APIs. To minimize false positives, the crash analysis module treats cases that are ambiguous or difficult to classify as misuse crashes. For example, if a crash is caused by passing a non-existent or error pointer to a library API, and the API does not employ defensive programming to check the validity of the input pointer, we still categorize this as a misuse crash. To verify the accuracy of the crash analysis module in detecting misuse crashes, we randomly sampled 10% of the misuse crash cases from each library for manual inspection. After reviewing these cases, we confirmed that all of them were indeed caused by the misuse of the API by the fuzz driver.

TABLE III: Summary of crash analysis results in CKGFuzzer

Library	UC	MC	RB	CB
c-ares	28	23	5	2(2)
cjson	14	12	2	0
curl	28	24	4	0
lcms	42	36	6	4
libpcap	22	18	4	0
libtiff	40	32	8	5
libvpx	8	8	0	0
zlib	17	15	2	0
Total	199	168	31	11(2)

UC: Unique Crashes, **MC**: Misuse Crashes, **RB**: Reported Bugs, **CB**: Confirmed Bugs (Fixed)

We also manually reviewed the crashes classified as reported bugs by the crash analysis module. In total, 11 real-world bugs were identified across three libraries, of which nine had not been previously reported. For the cases where the crash analysis module incorrectly classified a crash as a reported bug, the errors primarily stemmed from complex API call relationships within the libraries. The LLM had difficulty tracing deeply nested API calls when parsing the fuzz driver source code, which prevented it from fully understanding the input constraints and functionality of the nested APIs, ultimately leading to incorrect classifications.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

```
Function(cmsHANDLE h, const char* f)
{
     cmsIT8* it8 = (cmsIT8*) h;
     _cmsAssert(h != NULL);
     if(f == NULL)
         strcpy(it8->DoubleFormatter, DEFAULT_DBL_FORMAT);
     else
         strcpy(it8->DoubleFormatter, f);
 }
```

(a) A buffer overflow bug case in the `lcms`

```
Function(TIFF* tif)
{
     TIFFDirectory* td = &tif->tif_dir;
     if(td->width == 0 || td->length == 0 || td->depth == 0)
     {
         return 0;
     }
     return Function64(tif);
}
```

(b) An out of memory bug case in the `libtiff`

Fig. 3: Bugs found by CKGFuzzer

Our experimental results demonstrate that the crash analysis module effectively identifies the primary sources of crashes during fuzzing, reducing the manual review workload by an impressive 84.4%. This significantly improves the overall efficiency of bug triaging for software developers.

2) Case study: Figure 3a shows a buffer overflow bug case discovered by CKGFuzzer. In this code, the unsafe `strcpy` function is used without checking the length of the input string `f`. Since the data passed by the fuzz driver is randomly generated, the length of `f` is not controlled, and it may exceed the size of the `it8->DoubleFormatter` buffer. This leads to a buffer overflow, which is a severe security vulnerability that could potentially be exploited by attackers to overwrite memory and execute arbitrary code.

Figure 3b illustrates another bug case discovered by CKGFuzzer, which is an out-of-memory (OOM) issue. In this case, `*tif` refers to a TIFF file controlled by CKGFuzzer. The library function does not properly validate the fields such as width and length within the file. As a result, it returns an abnormal tile value. Other library APIs attempt to allocate memory based on the calculated tile size, which causes excessive memory allocation, ultimately leading to an OOM crash. Our case study demonstrates the effectiveness of CKGFuzzer in detecting real-world bugs. We will publicly disclose the bug cases we have discovered on our website.

V. Threats to Validity

A. External

The evaluation of CKGFuzzer was conducted using a set of eight widely-used open-source libraries. While these libraries cover a range of API usage patterns and functionalities, the generalizability of our approach to other libraries or domains remains uncertain. It is possible that CKGFuzzer's performance may vary when applied to libraries with different structures, API designs, or use cases. Another potential threat is bias introduced by the training data of the LLM used in CKGFuzzer. The LLM may favor over-represented patterns in its training set, which could affect the generation of API combinations or the crash analysis process. To mitigate this risk, we employed diverse prompts and knowledge sources to guide the LLM and reduce the impact of any biases.

B. Internal

A key internal threat to validity is the potential for LLM hallucinations during code generation. Incorrect or irrelevant code snippets may be generated, requiring multiple rounds of refinement or manual intervention to correct. This could affect the quality and functionality of the generated fuzz drivers. Additionally, the performance of CKGFuzzer depends heavily on the accuracy and completeness of the knowledge graph and API information extracted from the codebase. Incomplete or inaccurate knowledge may result in suboptimal API combinations or ineffective crash analysis.

VI. Discussion

A. Applicability to Different Programming Languages

CKGFuzzer was primarily evaluated on C-based libraries, which are widely used and have well-documented APIs. However, the applicability of CKGFuzzer to other programming languages, such as Rust, Java, or Python, remains an open question. Since each language has different API designs, CKGFuzzer may require adjustments to effectively handle these differences. Future work could explore the adaptation of CKGFuzzer to these languages, potentially broadening its scope and utility.

B. Quality of the Code Knowledge Graph

The code knowledge graph plays a central role in CKGFuzzer by guiding API combination generation and updating API combinations during coverage-guided mutation. The completeness and accuracy of the code knowledge graph are essential for generating effective fuzz drivers. In our experiments, libraries with more comprehensive code knowledge graph showed better coverage and higher fuzzing effectiveness. Improving the automation, construction, and validation of the code knowledge graph could further enhance the performance of CKGFuzzer by ensuring that it accurately reflects the library's API structure and usage patterns.

VII. Conclusion

In this paper, we presented CKGFuzzer, a novel LLM-driven fuzz testing framework enhanced by a code knowledge graph. CKGFuzzer automates fuzz driver generation and introduces several key features to improve fuzz testing effectiveness. These include a coverage-guided mutation strategy, which iteratively refines API combinations to explore new execution paths, a dynamic program repair mechanism that automatically resolves compilation errors in generated fuzz drivers, and a crash analysis module that helps identify the root causes of runtime failures. Our experiments on eight open-source projects demonstrate that CKGFuzzer achieves better code coverage and successfully detects 11 real-world bugs. The

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

coverage-guided mutation enables broader exploration of code paths, while the dynamic program repair and crash analysis modules significantly reduce manual intervention, making the fuzzing process more efficient and reliable. These results highlight the potential of leveraging LLMs and code knowledge graphs to advance automated fuzz testing.

Acknowledgments

This work was supported in part by the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), HUST CSE-HongXin Joint Institute for Cyber Security, HUST CSE-FiberHome Joint Institute for Cyber Security, and the Xiaomi Young Talents Program.

References

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] B. Blumbergs and R. Vaarandi, "Bbuzz: A bit-aware fuzzing framework for network protocol systematic reverse engineering and analysis," in *MILCOM 2017 - 2017 IEEE Military Communications Conference (MILCOM)*, 2017, pp. 707–712.

[3] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.

[4] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[5] J. Eom, S. Jeong, and T. Kwon, "Fuzzing javascript interpreters with coverage-guided reinforcement learning for llm-based mutation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 1656–1668. [Online]. Available: https://doi.org/10.1145/3650212.3680389

[6] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, "The robots are coming: Exploring the implications of openai codex on introductory programming," in *Proceedings of the 24th Australasian Computing Education Conference*, 2022, pp. 10–19.

[7] Google, "American fuzzy lop," https://github.com/google/AFL, 2024, accessed: 2024-10-18.

[8] H. Green and T. Avgerinos, "Graphfuzz: library api fuzzing with lifetime-aware dataflow graphs," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1070–1081. [Online]. Available: https://doi.org/10.1145/3510003.3510228

[9] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, Sep. 2024, just Accepted. [Online]. Available: https://doi.org/10.1145/3695988

[10] J. Hu, Q. Zhang, and H. Yin, "Augmenting greybox fuzzing with generative ai," *arXiv preprint arXiv:2306.06782*, 2023.

[11] L. Huang, P. Zhao, H. Chen, and L. Ma, "Large language models based fuzzing techniques: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2402.00350

[12] HuggingFace, "Baai/bge-small-en-v1.5." [Online]. Available: https://huggingface.co/BAAI/bge-small-en

[13] J. Jiang, H. Xu, and Y. Zhou, "Rulf: rust library fuzzing via api dependency graph traversal," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '21. IEEE Press, 2022, p. 581–592. [Online]. Available: https://doi.org/10.1109/ASE51524.2021.9678813

[14] Y. Jiang, J. Liang, F. Ma, Y. Chen, C. Zhou, Y. Shen, Z. Wu, J. Fu, M. Wang, S. Li, and Q. Zhang, "When fuzzing meets llms: Challenges and opportunities," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 492–496. [Online]. Available: https://doi.org/10.1145/3663529.3663784

[15] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "Inferfix: End-to-end program repair with llms," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1646–1656. [Online]. Available: https://doi.org/10.1145/3611643.3613892

[16] H. Joshi, J. Cambronero, S. Gulwani, V. Le, I. Radicek, and G. Verbruggen, "Repair is nearly generation: Multilingual program repair with llms," 2022. [Online]. Available: https://arxiv.org/abs/2208.11640

[17] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2123–2138. [Online]. Available: https://doi.org/10.1145/3243734.3243804

[18] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better zero-shot reasoning with role-play prompting," 2024. [Online]. Available: https://arxiv.org/abs/2308.07702

[19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[20] J. Li, B. Zhao, and C. Zhang, "Fuzzing: a survey," *Cybersecurity*, vol. 1, pp. 1–13, 2018.

[21] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[22] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[23] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model," 2023. [Online]. Available: https://arxiv.org/abs/2310.15657

[24] LlamaIndex, "Llamaindex documents." [Online]. Available: https://docs.llamaindex.ai/en/stable/

[25] LLVM, "Undefinedbehaviorsanitizer." [Online]. Available: https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

[26] LLVM, "libfuzzer – a library for coverage-guided fuzz testing," 2024, accessed: 2024-10-18. [Online]. Available: https://llvm.org/docs/LibFuzzer.html

[27] Y. Lyu, Y. Xie, P. Chen, and H. Chen, "Prompt fuzzing for fuzz driver generation," *arXiv preprint arXiv:2312.17677*, 2023.

[28] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[29] S. Mallissery and Y.-S. Wu, "Demystify the fuzzing methods: A comprehensive survey," *ACM Comput. Surv.*, vol. 56, no. 3, Oct. 2023. [Online]. Available: https://doi-org.libproxy.smu.edu.sg/10.1145/3623375

[30] V. J. Manes, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *arXiv preprint arXiv:1812.00140*, 2018.

[31] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.

[32] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," 2021. [Online]. Available: https://arxiv.org/abs/2108.09293

[33] Y. Pei, Y. Wei, C. A. Furia, M. Nordio, and B. Meyer, "Code-based automated program fixing," in *2011 26th IEEE/ACM International*

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio

Conference on Automated Software Engineering (ASE 2011), 2011, pp. 392–395.

[34] R. Qiu, W. W. Zeng, H. Tong, J. Ezick, and C. Lott, "How efficient is llm-generated code? a rigorous & high-standard benchmark," 2024. [Online]. Available: https://arxiv.org/abs/2406.06647

[35] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "Address-sanitizer: a fast address sanity checker," in *Proceedings of the 2012 USENIX Conference on Annual Technical Conference*, ser. USENIX ATC'12. USA: USENIX Association, 2012, p. 28.

[36] K. Serebryany, "Oss-fuzz: Google's continuous fuzzing service for open source software," 2017.

[37] M. Skelton, "Everything you need to know about ffuf," https://codingo.io/tools/ffuf/bounty/2020/09/17/everything-you-need-to-know-about-ffuf.html, 2024, accessed: 2024-10-18.

[38] E. Stepanov and K. Serebryany, "Memorysanitizer: Fast detector of uninitialized memory use in c++," in *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2015, pp. 46–55.

[39] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 724–735.

[40] M. Widl, "Test case generation by grammar-based fuzzing for model-driven engineering," in *Hardware and Software: Verification and Testing*, A. Biere, A. Nahir, and T. Vos, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 278–279.

[41] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," 2024. [Online]. Available: https://arxiv.org/abs/2405.04760

[42] L. Yan, Z. Zhang, G. Tao, K. Zhang, X. Chen, G. Shen, and X. Zhang, "Parafuzz: An interpretability-driven technique for detecting poisoned samples in nlp," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[43] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, and L. Zhang, "Whitefox: White-box compiler fuzzing empowered by large language models," *arXiv preprint arXiv:2310.15991*, 2023.

[44] C. Zhang, M. Bai, Y. Zheng, Y. Li, W. Ma, X. Xie, Y. Li, L. Sun, and Y. Liu, "Understanding large language model based fuzz driver generation," *arXiv e-prints*, pp. arXiv–2307, 2023.

[45] X. Zhang, C. Zhang, X. Li, Z. Du, B. Mao, Y. Li, Y. Zheng, Y. Li, L. Pan, Y. Liu *et al.*, "A survey of protocol fuzzing," *ACM Computing Surveys*, 2024.

[46] J. Zhao, Y. Rong, Y. Guo, Y. He, and H. Chen, "Understanding programs by exploiting (fuzzing) test cases," 2023. [Online]. Available: https://arxiv.org/abs/2305.13592

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:17:03 UTC from IEEE Xplore. Restrictio